МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
„КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ”
НАВЧАЛЬНО-НАУКОВИЙ КОМПЛЕКС
„ІНСТИТУТ ПРИКЛАДНОГО СИСТЕМНОГО АНАЛІЗУ”

ЛАБОРАТОРНА РОБОТА №6

з курсу: „Обробка надвеликих масивів даних”

виконав студент 5го курсу
групи КН-51мп
Сидько Н.С.

КИЇВ
2025

Метою досліджень по цій темі є отримання практичних навичок по техніках розширення функціональних можливостей словників, які входять до складу токенізаторів, шляхом введення токенів нових слів та знаків. В наведених завданнях використати тексти програмних кодів, створених на попередньому лабораторному занятті №1.

Завдання

1. Познайомитися з методами побудови словників і токенізаторів, які описані в даній інструкції, детально проаналізувати наведені приклади.

2. Вибрати самостійно кілька простих прикладів текстів для виконання експериментів.

3. Слідуючи пунктам завдання, описаного в цій інструкції, виконати експерименти з обраними прикладами текстів.

4. Представити повний письмовий звіт з виконаних досліджень

Хід роботи

Після вивчення основ побудови словників і принципів роботи токенізаторів можна перейти до практичних експериментів.

Додамо до списку всіх унікальних слів два токени <|unk|> і <|endoftext|> та протестуємо їх роботу:

```
import re

with open("the-verdict.txt", "r", encoding="utf-8") as f:
    raw_text = f.read()

preprocessed = re.split(r'([,.:;?_!"()\']|--|\s)', raw_text)
preprocessed = [item.strip() for item in preprocessed if item.strip()]

all_tokens = sorted(list(set(preprocessed)))
all_tokens.extend(["<|endoftext|>", "<|unk|>"])
vocab = {token:integer for integer,token in enumerate(all_tokens)}
print(len(vocab.items()))

for i, item in enumerate(list(vocab.items())[-5:]):
    print(item)
```

```
1132
('younger', 1127)
('your', 1128)
('yourself', 1129)
('<|endoftext|>', 1130)
('<|unk|>', 1131)
```

Рис.1 Результат застосування токенів <|unk|> і <|endoftext|>

У результаті ми впевнилися, що токени були успішно вписані у словник.

<|unk|> - це токен, що позначає невідомий або рідкісний символ чи слово, якого немає в словнику моделі.

<|endoftext|> - цей токен використовується для позначення кінця тексту або завершення послідовності при навчанні та генерації тексту.

Змінимо попередній токенізатор на новий клас SimpleTokenizerV2 з підтримкою раніше вивчених токенів:

```
class SimpleTokenizerV2:
    def __init__(self, vocab):
        self.str_to_int = vocab
        self.int_to_str = { i:s for s,i in vocab.items()}

    def encode(self, text):
        preprocessed = re.split(r'([,.:;?_!"()\']|--|\s)', text)
        preprocessed = [
        item.strip() for item in preprocessed if item.strip()
        ]
        preprocessed = [item if item in self.str_to_int            #1
        else "<|unk|>" for item in preprocessed]
        ids = [self.str_to_int[s] for s in preprocessed]
        return ids

    def decode(self, ids):
        text = " ".join([self.int_to_str[i] for i in ids])
        text = re.sub(r'\s+([,.:;?!"()\'])', r'\1', text)    #2
        return text
```

Рис.2 Клас SimpleTokenizerV2

Далі проведемо тестування нового токенізатора:

```
text1 = "Hello, do you like tea?"
text2 = "In the sunlit terraces of the palace."
text = " <|endoftext|> ".join((text1, text2))
print(text)

tokenizer = SimpleTokenizerV2(vocab)

encode = tokenizer.encode(text)
print("Кодування: ", encode)
print("Декодування: ", tokenizer.decode(encode))
```

```
Hello, do you like tea? <|endoftext|> In the sunlit terraces of the palace.
Кодування:  [1131, 5, 355, 1126, 628, 975, 10, 1130, 55, 988, 956, 984, 722, 988, 1131, 7]
Декодування:  <|unk|>, do you like tea? <|endoftext|> In the sunlit terraces of the <|unk|>.
```

Рис.3 Результат роботи нового токенізатора

Результат демонструє, що токенізатор правильно кодує та відновлює текст, зберігаючи структуру і пунктуацію, водночас обробляючи невідомі слова.

Дослідимо схему токенізації через кодування пар байтів, яка використовується в таких LLM, як GPT-2, GPT-3 і в моделі, яка використовується в ChatGPT:

```
from importlib.metadata import version
import tiktoken
print("tiktoken version:", version("tiktoken"))

tokenizer = tiktoken.get_encoding("gpt2")
text = ("Hello, do you like tea? <|endoftext|> In the sunlit terraces of
someunknownPlace.")

encode = tokenizer.encode(text, allowed_special={"<|endoftext|>"})

print("Кодування: ", encode)
print("Декодування: ", tokenizer.decode(encode))
```

```
tiktoken version: 0.12.0
Кодування:  [15496, 11, 466, 345, 588, 8887, 30, 220, 50256, 554, 262, 4252, 18250, 8812, 2114, 286, 617, 34680, 27271, 13]
Декодування:  Hello, do you like tea? <|endoftext|> In the sunlit terraces of someunknownPlace.
```

Рис.4 Результат роботи токенізатора через кодування байтів

Порівнюючи токенізатори, можна сказати, що простий токенізатор SimpleTokenizerV2 працює з обмеженим словником і замінює невідомі слова, яких немає у словнику (наприклад, Hello або someunknownPlace), на спеціальний токен <|unk|>. Через це частина змісту тексту втрачається, і декодований текст відновлюється не повністю.

На відміну від цього, метод Byte Pair Encoding (BPE), який використовується у токенізаторі tiktoken для моделі GPT-2, здатний розбивати будь-яке нове слово на менші підслова або символи. Це дозволяє обробляти будь-який текст без втрат інформації та точно відновлювати вихідний текст при декодуванні.

Словник SimpleTokenizerV2 формується лише на основі унікальних слів конкретного набору даних, тоді як BPE використовує великий попередньо створений словник, який охоплює багато слів і підслів. Це робить BPE більш

гнучким і універсальним для сучасних мовних моделей, забезпечуючи кращу точність та семантичну цілісність тексту.

Висновок:

Під час виконання лабораторної роботи було досліджено способи покращення токенізаторів та розширення їхніх функціональних можливостей. Було створено оновлений словник із додаванням спеціальних токенів, таких як <|endoftext|>, що дозволяє токенізатору правильно розділяти текстові фрагменти з різних джерел. Розроблено вдосконалений клас SimpleTokenizerV2, який здійснює кодування та декодування тексту без помилок і коректно обробляє спеціальні токени. Також проведено порівняння цього токенізатора з методом Byte Pair Encoding (BPE), який використовується в сучасних мовних моделях. Виявлено, що BPE є більш універсальним та точним підходом, оскільки дозволяє працювати з будь-якими словами, розбиваючи їх на менші підслова без втрати змісту. В результаті виконаної роботи отримано глибше розуміння процесів формування словників та токенізаторів, а також того, як різні підходи впливають на точність і гнучкість обробки текстових даних.